Step 0 — Split train / dev / test e protocollo di valutazione

Obiettivo

Prima di misurare qualsiasi cosa (baseline compresa) e prima di generare dati, costruire e **congelare** un dataset di valutazione nostro, senza LLM-AggreFact:

- **train**: l'unico insieme da cui si addestra e da cui si generano nuovi dati;
- **dev**: l'unico insieme su cui si prendono decisioni (soglia, temperatura, checkpoint, esperimenti);
- **test**: si guarda una volta sola, alla fine, insieme ai test esterni.

LLM-AggreFact **non entra in nessuna decisione**: diventa un test esterno, valutato *zero-shot*, come le voci della sua leaderboard.

Perché questo ordine

1. **AggreFact diventa confrontabile con la leaderboard.** Fino a oggi soglia e temperatura sono state fittate sul dev di AggreFact: i numeri sono "dev-tuned". Le voci della leaderboard sono zero-shot. Se tutto si sceglie sul nostro dev, il risultato su AggreFact misura davvero la generalizzazione.
2. **Baseline e fine-tuning con lo stesso protocollo.** Il modello di partenza e quello fine-tunato prendono soglia e temperatura dallo stesso dev, con la stessa procedura.
3. **Niente fughe dai dati generati.** La generazione con LLM (02b) parte solo da righe di train. Se gli split non fossero congelati prima, documenti di test potrebbero finire nel training in forma modificata (es. con l'evidenza cancellata).

1. I blocchi di valutazione

Blocco	Contenuto	A cosa serve	Quando si guarda
dev_select	~60% del dev di ciascuna fonte di training	scegliere soglia, temperatura, checkpoint, esperimenti	sempre
dev_confirm	~40% del dev (documenti diversi)	confermare la scelta senza averla ottimizzata lì	una volta per fase
test_id (in distribuzione)	test delle stesse fonti del training	quanto il modello ha imparato	alla fine, una volta
test_ood (fuori dominio)	fonti **mai** usate in train né in dev	quanto generalizza	alla fine, una volta
AggreFact test	11 fonti, 29.320 righe	confronto con la leaderboard (zero-shot)	alla fine, una volta
Oblio	JevBench pubblico, Kev-v9 clean, typed-decisions test	capacità di decisione generali	a ogni esperimento (non servono per scegliere)

Il dev è diviso in due pool (select e confirm) come nel protocollo che ha prodotto il risultato di Jev in questa repo: dopo decine di esperimenti sul pool di selezione, il pool di conferma dice se il miglioramento è reale o è sovra-adattamento al dev.

2. Da dove viene ogni split, per fonte

Regola generale: si usano i test **ufficiali** quando sono etichettati; altrimenti si divide un insieme etichettato **per documento** (`doc_id`).

Fonte	train	dev (select + confirm)	test_id
FEVER	train ufficiale	`paper_dev`	`paper_test` (ufficiale, etichettato)
ANLI	train R1–R3	dev R1–R3	test R1–R3 (ufficiale)
VitaminC	train	validation	test (ufficiale)

Fonte	train	dev (select + confirm)	test_id
HoVer	train	metà del dev ufficiale, per documento	altra metà del dev ufficiale (il test ufficiale non ha etichette)
AVeriTeC	train	metà del dev ufficiale (~230 righe)	altra metà (il test ufficiale non ha etichette)
MiniCheck C2D/D2C	80% dei documenti	10%	10% (split unico in origine, diviso per `doc_id`)
SummEdits	80% dei documenti	10%	10% (idem; i riassunti dello stesso documento restano insieme)
FactKG (solo se trasformato in prosa)	train	dev	test (ufficiali)

Fonti **solo di test fuori dominio** (mai in train né in dev):

Fonte	Dominio	Note
SciFact	biomedico	toglierla dalle ricette di training (oggi è in `core`)
Climate-FEVER	clima, claim reali	evidenza a livello di frase
HealthVer, COVID-Fact	salute	
HaluBench	QA in finanza, medicina, DROP	la parte RAGTruth si riporta a parte (è anche una fonte di AggreFact)
HalluMix	multi-documento, documenti lunghi	controllare le sovrapposizioni con AggreFact
LLM-Oasis (test annotato a mano)	Wikipedia	se disponibile nella variante con evidenza

Esclusi del tutto: RAGTruth (98% dei documenti in comune con AggreFact), WiCE (fonte di AggreFact; solo per un esperimento dedicato, mai nei test), LLM-AggreFact dev e test dal training.

3. Dimensioni

Insieme	Righe per fonte	Perché
dev_select	fino a 600, bilanciate 50/50	errore della BAcc per fonte di ~$\pm$3 punti, sufficiente per scegliere
dev_confirm	fino a 400, bilanciate	conferma
test_id	fino a 2.000, bilanciate	errore di ~$\pm$2 punti per fonte
test_ood	tutto ciò che è disponibile (SciFact ~1.250 coppie fra train e dev ufficiali, Climate-FEVER ~1.500 claim, HaluBench ~15.000)	non si addestra su queste fonti, quindi si possono usare per intero

Con 7 fonti di training: dev $\approx$ 7.000 righe, test_id $\approx$ 12.000 righe. La baseline su dev + test richiede poche ore di inferenza per modello.

4. Regole contro le fughe di informazione

1. **Per documento**: nessun `doc_id` in più di uno split. Le righe di train che condividono un documento con dev o test si tolgono dal train (già oggi `make_mix.py` lo fa per il val: 37.702 righe in ANLI, 4.826 in FEVER).
2. **Quasi-duplicati fra split**: stesso controllo a 13-grammi usato contro AggreFact, applicato anche fra il nostro train e i nostri dev/test (es. pagine Wikipedia quasi identiche in FEVER e VitaminC).

3. **Decontaminazione contro tutti i test esterni**: LLM-AggreFact (dev e test), tutti i dataset di test_ood, i benchmark di oblio.
4. **Generazione solo dal train**: i task di 02b e le domande di replay di 02c partono solo da righe di train. Le coppie generate ereditano il `doc_id`, quindi restano nel train.
5. **Etichette originali in dev e test**: i dati generati con LLM vanno solo nel train. Se servisse misurarli, in un blocco "test sintetico" separato, mai mescolato con test_id.

5. Come si aggregano i risultati

Non si mescolano le righe di test diversi in un unico calcolo: le fonti hanno dimensioni molto diverse e la più grande dominerebbe.

1. BAcc **per fonte**.
2. Media non pesata **per blocco**: test_id, test_ood, AggreFact (media sulle sue 11 fonti, come la leaderboard).
3. **Numero unico** = media dei tre blocchi.
4. LLM-AggreFact riportato **anche da solo**, nel formato della leaderboard.

Esempio di report (numeri illustrativi):

Modello	test_id	test_ood	AggreFact (zero-shot)	Media dei blocchi
Winnow-12B baseline	82,0	74,5	77,0	77,8
Winnow-12B fine-tunato	88,5	77,0	78,6	81,4

6. Soglia: principale e secondaria

- **Principale**: soglia e temperatura fittate su `dev_select`, confermate su `dev_confirm`, applicate a tutti i test. Su AggreFact il risultato è zero-shot.
- **Secondaria, dichiarata**: soglia fittata sul dev di AggreFact, solo per confrontarsi con i numeri già prodotti in questa repo (Winnow-BF16 77,42; JPT-9B 76,71). Il dev di AggreFact non si usa per nient'altro.

Possibile effetto: una soglia scelta sul nostro dev (molta Wikipedia e NLI) può essere un po' meno adatta ai riassunti di AggreFact; si potrebbero perdere da qualche decimo a ~1 punto rispetto alla secondaria. L'AUC non dipende dalla soglia e non cambia. La differenza fra le due va riportata: misura quanto il modello dipende dal dominio per la sua calibrazione.

7. Il dev di AggreFact

Per restare zero-shot, il dev di AggreFact non entra in nessuna decisione: né soglia, né checkpoint, né scelta fra esperimenti. Se lo si guarda come monitoraggio, va dichiarato, sapendo che guardarlo spesso lo trasforma di fatto in un secondo dev.

8. Output e congelamento

```
data/finetune/splits/
  train/<fonte>.jsonl
  dev_select/<fonte>.jsonl
  dev_confirm/<fonte>.jsonl
  test_id/<fonte>.jsonl
  test_ood/<fonte>.jsonl
  manifest.json          # righe per split e fonte, hash SHA-256 di ogni file, regole, data
```

Dopo il congelamento i file di dev e test non si modificano più. Ogni risultato riporta l'hash del manifest.

9. Codice

Esiste: `data_prep/build_pairs.py` (coppie da 11 fonti, con `--splits train dev test`), `data_prep/make_mix.py` (decontaminazione contro AggreFact, split train/val per documento).

Da scrivere (`data_prep/make_splits.py` o estensione di `make_mix.py`):

- split a tre vie con le regole di §2 (test ufficiali, divisione per `doc_id` dove serve, dev in select/confirm);
- builder per le fonti di test_ood (SciFact c'è già; Climate-FEVER, HealthVer, COVID-Fact, HaluBench, HalluMix);
- decontaminazione estesa (§4) e manifest con hash.

Le ricette del mix di training (02e) leggono poi solo `splits/train/`.

Criteri per passare allo step successivo

- `manifest.json` scritto; nessun `doc_id` condiviso fra split; decontaminazione eseguita contro tutti i test esterni.
- Numero di righe per fonte e per split registrato.